

Lecture 13

Deep Learning (Agent Perspective)

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool



From Learning to Structure

Last lecture:

- ▶ Define loss
- ▶ Use gradients

Today:

- ▶ How architecture shapes what we can learn

Fully Connected Networks

In a fully connected layer:

Every input connects to every output.

Assume:

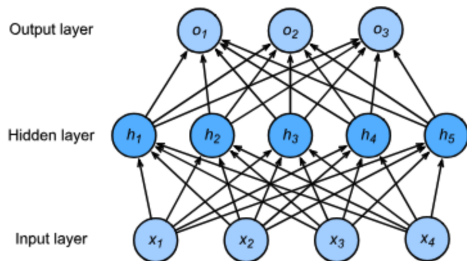
- ▶ Input $x \in \mathbb{R}^d$
- ▶ Output $h \in \mathbb{R}^m$
- ▶ Weight matrix $W \in \mathbb{R}^{m \times d}$

Matrix form:

$$h = Wx$$

Expanded form:

$$h_i = \sum_{j=1}^d W_{ij}x_j$$



Problem: Images Are Large

Example:

Image size: 1000×1000 pixels

That is 1 million inputs.

Fully connected layer would need millions of parameters.



Observation: Translation Invariance and Locality

In images:

- ▶ Nearby pixels are correlated
- ▶ Patterns repeat across locations

Two Principles:

- ▶ Translation Invariance
- ▶ Locality



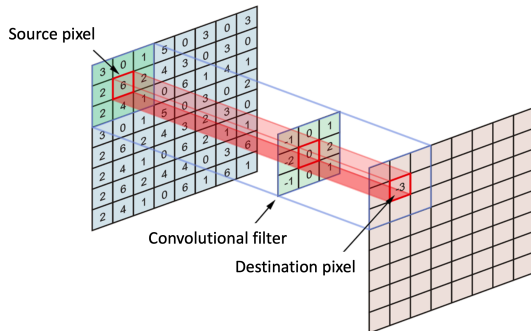
We should use this structure.

The Idea of Convolution

Instead of connecting everything:
Use a small filter (kernel) that slides across
the image.

Same filter used everywhere.

2D image convolution: Doing convolution
using different filters/kernels



2-D Convolution Layer

Input		Kernel		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25$$

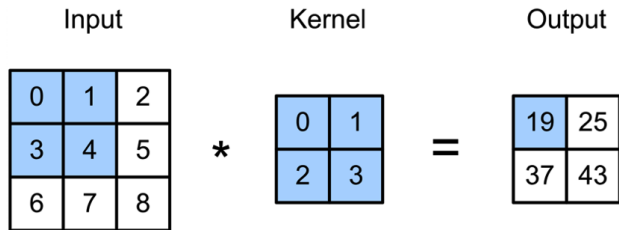
$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37$$

$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43$$

$$Y(i,j) = \sum_{k=0}^{k_h-1} \sum_{l=0}^{k_w-1} I(i+k, j+l) H(k,l)$$

(I : input image and H : kernel)

2-D Convolution Layer



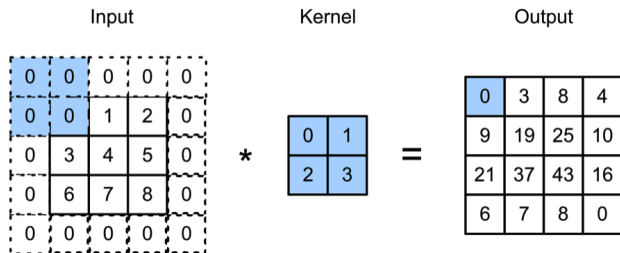
- ▶ X : $n_h \times n_w$ input matrix
- ▶ W : $k_h \times k_w$ kernel matrix
- ▶ b : scalar bias
- ▶ Y : $(n_h - k_h + 1) \times (n_w - k_w + 1)$ output matrix

$$Y = X \star W + b$$

- ▶ W and b are learnable parameters

Convolution Layer - Padding

Padding adds rows/columns around input



$$0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$$

Convolution Layer - Padding

- ▶ Padding p_h rows and p_w columns, the output shape becomes

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

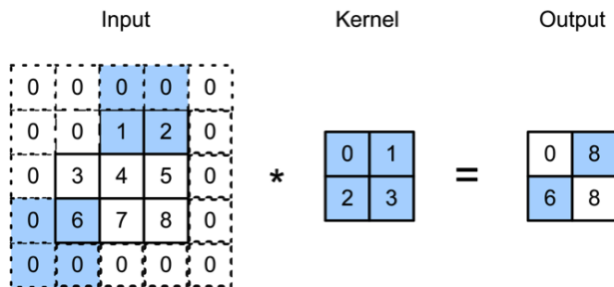
- ▶ A common choice is

$$p_h = k_h - 1, \quad p_w = k_w - 1$$

- ▶ If k_h is odd: pad $p_h/2$ on both sides
- ▶ If k_h is even: pad $\lceil p_h/2 \rceil$ on top, $\lfloor p_h/2 \rfloor$ on bottom

Convolution Layer - Stride

Stride is the #rows/#columns per slide



Strides of 3 and 2 for height and width

$$0 \times 0 + 0 \times 1 + 1 \times 2 + 2 \times 3 = 8$$

$$0 \times 0 + 6 \times 1 + 0 \times 2 + 0 \times 3 = 6$$

Convolution Layer - Stride

- ▶ Given stride s_h for height and stride s_w for width, the output shape is

$$\left\lfloor \frac{n_h - k_h + p_h}{s_h} \right\rfloor + 1 \times \left\lfloor \frac{n_w - k_w + p_w}{s_w} \right\rfloor + 1$$

- ▶ With $p_h = k_h - 1$ and $p_w = k_w - 1$

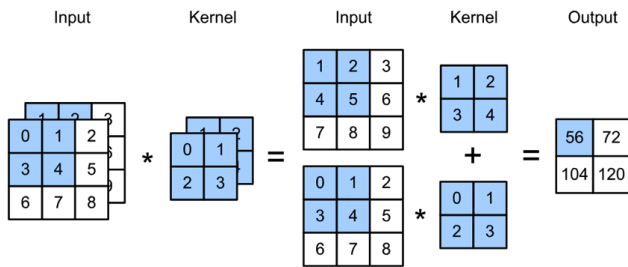
$$\left\lfloor \frac{n_h + s_h - 1}{s_h} \right\rfloor \times \left\lfloor \frac{n_w + s_w - 1}{s_w} \right\rfloor$$

- ▶ If input height/width are divisible by strides:

$$\frac{n_h}{s_h} \times \frac{n_w}{s_w}$$

Convolution Layer - Multiple Input Channels

Have a kernel for each channel, and then sum results over channels



$$\begin{aligned} & (1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) \\ & + (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) \\ & = 56 \end{aligned}$$

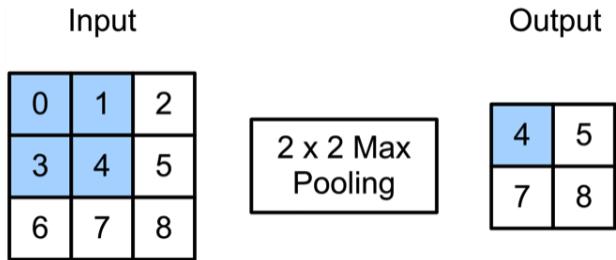
Convolution Layer - Multiple Input Channels

- ▶ X : $c_i \times n_h \times n_w$ input
- ▶ W : $c_i \times k_h \times k_w$ kernel
- ▶ Y : $m_h \times m_w$ output

$$Y = \sum_{c=1}^C X^{(c)} \star W^{(c)}$$

Pooling

Returns the maximal value in the sliding window



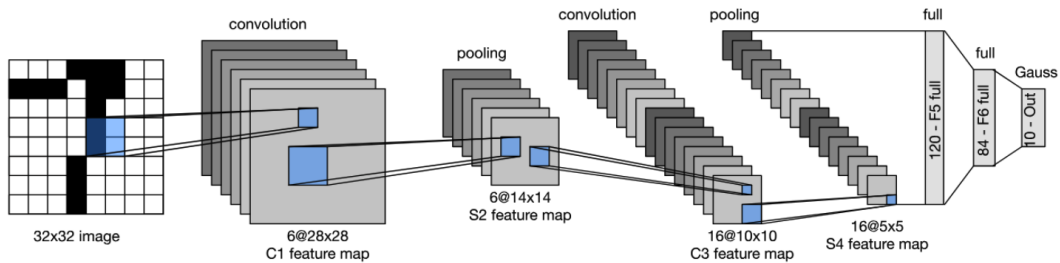
2-D Max Pooling
 $\max(0, 1, 3, 4) = 4$

Pooling

- ▶ Pooling layers have similar padding and stride as convolutional layers
- ▶ No learnable parameters
- ▶ Apply pooling for each input channel to obtain the corresponding output

channel $\#$ output channels = $\#$ input channels

LeNet



LeNet - Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader

# -----
# LeNet (LeNet-5 style)
# -----
class LeNet(nn.Module):
    """
    LeNet-5 style network for 1x28x28 (MNIST).
    Architecture (common modern variant):
    Conv(1->6, 5x5) -> ReLU -> AvgPool(2)
    Conv(6->16, 5x5) -> ReLU -> AvgPool(2)
    Flatten
    FC 16*4*4 -> 120 -> ReLU
    FC 120 -> 84 -> ReLU
    FC 84 -> num_classes
    """
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=0)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.pool = nn.AvgPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(16 * 4 * 4, 120) # for 28x28 input
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 28->24->12
        x = self.pool(F.relu(self.conv2(x))) # 12->8->4
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

Parameter Sharing

Fully connected:

- ▶ Different weights for every position

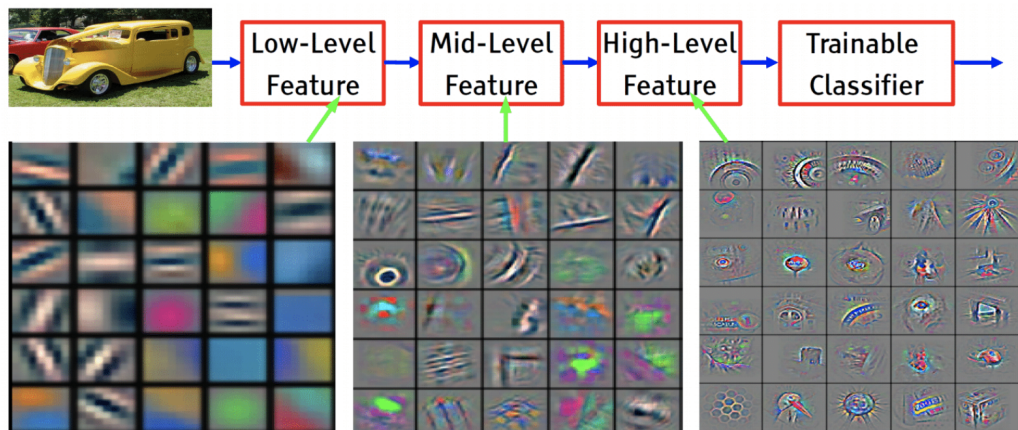
Convolution:

- ▶ Same weights reused across space

Translation Invariance

If an object moves slightly, the detector should still recognize it.
Convolution encodes this assumption.

Hierarchy in CNNs



Layers build hierarchy:
Pixels $\rightarrow$ Edges $\rightarrow$ Shapes $\rightarrow$ Objects
Each layer increases abstraction.

Limitations of Convolution

Convolution assumes:

- ▶ Local interactions dominate

But in language:

- ▶ Words far apart influence each other

Static Connectivity

Convolution uses fixed local connections.
Connections do not change based on input.

Neural Networks in Agents

For perception:

- ▶ Images → CNN

Architecture depends on data structure.

Key Takeaways

- ▶ Fully connected layers ignore structure.
- ▶ Convolution encodes locality.
- ▶ Parameter sharing reduces complexity.
- ▶ Inductive bias improves efficiency.
- ▶ Convolution has limitations.

Next Lecture

Next:

- ▶ Attention
- ▶ Transformers